

Interview Question Bank

Likely questions by round, with model answers

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 3 of 8

Contents

How to use this	3
Technical round	4
Questions the seniors were actually asked [ASKED]	4
Resume-driven technical questions (know every one of these)	5
How to connect the dots (the seniors' key tip)	7
Managerial round	8
HR round	9
HR round mindset	9
The night-before shortlist	10

How to use this

This is a bank of the questions most likely to come up, grouped by round, each with a short model answer **at your level**. Answers here are deliberately concise - the goal is that you can say the core idea correctly and conversationally, then let them dig deeper. **Coding answers live in part 4** (DSA & CS Fundamentals); here we cover the concepts.

Two reminders that apply to every answer:

- **Keep it short, then let them ask.** Do not empty your whole brain on the first question.
- **If you don't know, say so honestly** and offer to reason it out or look it up. A senior got selected doing exactly that.

The questions marked **[ASKED]** were reported by the placed seniors as questions they actually got. Prioritise those.

Technical round

Questions the seniors were actually asked [ASKED]

Introduce yourself (technical version). -> See part 0. Lead with what you build, name DokLink (live on Play Store), list your real stack, end on wanting to learn enterprise engineering.

Tell me something that is not on your resume. -> Have one genuine, human thing ready (part 0). Do not turn it into another technical brag.

Explain your project. -> Lead with DokLink. Give the 30-second overview (emergency app, find and book hospital beds in real time, I built app + backend + servers, it's on the Play Store), then stop and let them pick what to dig into. Have the race-condition story ready.

What are hashmaps? -> A hashmap stores **key-value pairs** and lets you look up a value by its key in **O(1) average time**. It works by running the key through a **hash function** that decides where to store it. In C++ it's `unordered_map`; in JavaScript it's a plain object or `Map`. I use them whenever I need fast lookups by a unique key - like caching or counting. (Code and collision handling in part 4.)

Write code to insert and delete a node in a doubly linked list. -> Full C++ code in part 4. Key idea to say: a doubly linked list node has **data**, a **next** pointer, and a **prev** pointer; insertion/deletion is just careful pointer rewiring, and you must handle the head/tail edge cases.

Write BFS and DFS. -> Full C++ code in part 4. One-liners: **BFS** explores level by level using a **queue**; **DFS** goes as deep as possible first using a **stack** (or recursion). Both need a **visited** set to avoid cycles.

Binary search. -> Full code in part 4. Idea: on a **sorted** array, repeatedly check the middle and throw away half. **O(log n)**.

Floor and ceiling type concepts. -> Given a sorted array and a value x: the **floor** is the largest element $\leq x$, the **ceiling** is the smallest element $\geq x$. Both are found with a modified binary search. (Part 4.)

What is inheritance? -> An OOP concept where a class (child) **reuses and extends** another class (parent), inheriting its properties and methods. It models an "is-a" relationship - a **Car** is a **Vehicle**. It promotes code reuse. (Four pillars and example in part 4.)

Questions on ACID properties. -> ACID = **Atomicity, Consistency, Isolation, Durability** - the four guarantees a database transaction gives. This is directly relevant to my DokLink bed-booking, which relies on atomic transactions. (Full breakdown in part 4.)

What is exception handling? -> A structured way to deal with errors at runtime without crashing: you **try** risky code, **catch** the error if it happens, and handle it gracefully. (Examples in part 4.)

Scenario-based database design. -> They describe a business situation and ask you to design tables and relationships. Approach: identify the **entities** (things: users, orders, products), give each a table with a **primary key**, connect them with **foreign keys**, and decide **one-to-many** vs **many-to-many** relationships. (Full worked example in part 4.)

Resume-driven technical questions (know every one of these)

Everything below is on your resume, so it is fair game. Short answers to make your own.

Web / full-stack

What is REST / a REST API? -> REST (Representational State Transfer) is a style for building web APIs over HTTP. Resources have URLs, and you use HTTP methods - GET (read), POST (create), PUT/PATCH (update), DELETE (remove). It's **stateless**: each request carries everything the server needs. My DokLink backend is a REST API built with Django REST Framework.

What is an API? -> Application Programming Interface - a defined way for two pieces of software to talk. My mobile app talks to my backend through a REST API.

Difference between React and Next.js? -> React is a library for building user interfaces that runs in the browser. Next.js is a framework built on React that adds server-side rendering, routing, and backend capabilities. I use Next.js when SEO and load speed matter (e-commerce), plain React inside it for interactivity.

What is server-side rendering and why does it matter? -> The page's HTML is built on the server and sent complete, so search engines and users get content immediately. It's better for SEO and first-load speed than client-side rendering, which sends a blank page that fills in later.

What is React Native? Why over Flutter? -> React Native builds real mobile apps using JavaScript/React. I chose it over Flutter for DokLink because it reused my existing React skills, let me move fast as a solo developer, and has a huge ecosystem; Flutter's Dart was a learning cost I didn't need. (Full reasoning in part 2.)

What is TypeScript and why use it? -> JavaScript with static types. It catches type errors before the code runs, which prevents a whole class of bugs and makes big codebases safer to change.

Backend / databases

Difference between SQL and NoSQL / PostgreSQL and MongoDB? -> SQL databases (PostgreSQL) are relational - structured tables with fixed schemas and strong consistency and transactions. NoSQL (MongoDB) stores flexible documents, good for unstructured or rapidly-changing data. I chose PostgreSQL for DokLink because bookings need transactions and relational integrity.

What is an ORM? -> Object-Relational Mapping - a layer that lets you work with the database using code objects instead of raw SQL. Django's ORM and Prisma are examples I've used.

What is indexing in a database? -> An index is a data structure (usually a B-tree) that makes lookups fast, like a book's index - instead of scanning every row, the database jumps straight to matching rows. I used indexed geo-lookups so DokLink finds nearby hospitals quickly. The trade-off is that indexes cost extra storage and slightly slow down writes.

What is a race condition? How did you handle it? -> When two operations access shared data at the same time and the result depends on timing. In DokLink, two users booking the last bed simultaneously. I fixed it with atomic transactions and row-level locking so only one wins and the count can't go negative. (Your best story - see part 2 and 4.)

What is Redis? -> An in-memory data store, extremely fast, used for caching, sessions, and rate limiting. It's on my resume for fast, temporary data.

Security (all on your resume)

What is JWT? -> JSON Web Token - a signed token the server issues after login. The client sends it with each request; the server verifies the signature instead of storing a session. It's stateless. I use it in DokLink.

What is OTP? -> One-Time Password - a short code, valid once and briefly, sent to verify a user really controls a phone or email. I use it to verify identity in DokLink.

What is HMAC-SHA256, and where did you use it? -> A cryptographic signature using a shared secret and the SHA-256 hash function. After a Razorpay payment, I verify the returned signature with HMAC-SHA256 to prove the confirmation genuinely came from Razorpay and wasn't forged.

What is XSS? -> Cross-Site Scripting - an attacker injects malicious JavaScript into a page so it runs in other users' browsers. Prevented by escaping/sanitising output and using safe frameworks.

What is CSRF? -> Cross-Site Request Forgery - a malicious site tricks a logged-in user's browser into making an unwanted request using their session. Prevented with anti-CSRF tokens and same-site cookies.

What is rate limiting? -> Capping how many requests a user or IP can make in a time window, to stop brute-force attacks and bot abuse. I used multi-tier rate limiting on A Fashions.

What is SSL/TLS / HTTPS? -> The encryption that secures traffic between browser and server so nobody can read or tamper with it. I automated SSL certificates with Certbot on my VPS deployments.

What is role-based access control? -> Giving users permissions based on their role (patient, hospital staff, admin) so each only accesses what they should.

DevOps / infrastructure

What is Docker / a container? -> Docker packages an app with everything it needs to run into a container, so it behaves the same on my machine, a server, or anywhere. It removes "works on my machine" problems.

Docker vs a virtual machine? -> A VM virtualises a whole operating system (heavy); a container shares the host OS kernel and only packages the app (lightweight and fast to start).

What is Nginx / a reverse proxy? -> Nginx is a web server I use as a reverse proxy - it sits in front of my app, receives all incoming traffic, forwards it to the app, and handles HTTPS. It can also serve static files and balance load.

What is CI/CD? -> Continuous Integration / Continuous Deployment - automatically testing and deploying code when it changes, so releases are frequent, safe, and repeatable.

What is a VPS? -> Virtual Private Server - a server I fully control, where I set up the OS, web server, and app myself. I chose it over managed hosting for control and cost.

AI / ML (from FloatChat)

What is an LLM? -> Large Language Model - an AI model trained on huge amounts of text that can understand and generate language. I used Mistral 7B in FloatChat.

What is RAG? -> Retrieval-Augmented Generation - retrieve relevant context first, then let the model answer grounded in that real information instead of relying on its memory. Reduces wrong answers.

What is a vector database / embedding? -> An embedding turns text into numbers capturing its meaning; a vector database stores these so you can search by **similarity of meaning** rather than exact words.

What is ETL? -> Extract, Transform, Load - pull data from a source, reshape it, load it into your database. I built ETL pipelines to move NetCDF ocean files into PostgreSQL.

How to connect the dots (the seniors' key tip)

The seniors said: **connect the dots - tie technical answers back to your projects.** Whenever you explain a concept, land it on where you actually used it. "A JWT is a signed token for stateless auth - I use exactly this in DokLink's login." That does two things: it proves you didn't just memorise a definition, and it steers them toward your projects, where you are strongest. Practise ending concept answers with "...and I used this in [project]."

Managerial round

The managerial round is about **how you work, how you think, and whether you fit**. Full model answers are in part 0 - here are the questions and the one-line angle for each.

Introduce yourself (managerial version). -> Focus on how you work: ownership, learning by building, teamwork. (Part 0.)

[ASKED] Where do you see yourself in 5 years? -> Grow from solo builder to a trusted engineer in a real org; first two years as a Scholar learning + M.Tech; then real ownership, later maybe leading a technical area. Anchor it in the program's structure. (Part 0.)

[ASKED] Why SAP? -> Scale (runs the world's biggest businesses), it's the full-stack, real-business-problem work I already love, and the STAR program specifically. (Part 0 and 1.)

[ASKED] Why STAR? -> It matches how you learn - real work + rotations + sponsored M.Tech + a two-year depth. Say it with genuine enthusiasm. (Part 1 has the full model answer.)

[ASKED] How will you manage work and college / the M.Tech? -> You already juggle final-year study with running DokLink and client work; the program is more structured than that. Plan the week, protect deep-work time, flag problems early. (Part 0.)

[ASKED] Strengths and weaknesses. -> Strengths: ownership (DokLink end to end) and fast self-teaching. Weakness: working solo, you sometimes over-engineer before asking for the simplest path - and you're actively fixing it; a team would help. (Part 0.)

What did you learn in your internship / experience? -> Not what you did - what you learned: production is far harder than class projects (reliability, the race condition, running servers, recovering from failure), and to make trade-offs consciously. (Part 0.)

Tell me about a challenge you faced and how you solved it. -> The DokLink race condition is perfect: real problem (two users, last bed), real stakes (healthcare), a clear solution (atomic transactions + locking + auto-expiry), and a lesson (correctness over cleverness). Use the STAR structure: Situation, Task, Action, Result.

How do you handle working under pressure / alone? -> Be honest: you've run a real product alone, so you've learned to prioritise, keep things reliable, and stay calm when something breaks in production. Give the example of fixing a live issue.

Are you a team player? Give an example. -> The client projects (with designers and founders) and the research papers (a team over months) show you collaborate, take feedback, and communicate - even though you often work solo.

HR round

The HR round checks **attitude, honesty, communication, and fit**. Be warm, natural, and genuine.

Tell me about yourself. -> A lighter, more personal version of the intro - who you are, what you love about building software, a bit of personality. Not a tech dump.

Why should we hire you? / Why are you a good fit? -> “Because I already do a version of this job - I build and run real production software, alone, end to end. I bring proof that I ship, not just study. And I’m genuinely excited to learn enterprise engineering, which is exactly what the Scholar program teaches.” Confidence + humility.

What do you know about SAP? -> The one-paragraph summary from part 1: world-leading enterprise/ERP software, runs most large businesses, modern products on the HANA in-memory database, moving to the cloud, embedding AI via Joule.

What are your hobbies / tell us something about you. -> Be a real person. Have one or two genuine answers ready. This is rapport, not a test.

Are you willing to relocate / commit to the 2-year program? -> Yes, clearly and confidently. Show you understand it’s a two-year journey and you’re committed to it.

Do you have any offers / are you interviewing elsewhere? -> Be honest but keep SAP as your clear priority: “SAP and the STAR program are genuinely my top choice because of the learning and the M.Tech - it’s the opportunity I most want.”

[ASKED] Do you have any questions for us? -> ALWAYS yes. Ask about rotations, the first months of learning, what makes a Scholar excel, or the team’s day-to-day. Optionally share a learning from your own work. (Full list in part 0.)

HR round mindset

HR is often where people relax and get sloppy, or freeze up. Neither. Stay warm, honest, and conversational. Smile. They’re checking that you’re someone the team will enjoy working with and that your enthusiasm is real. It is - let it show.

The night-before shortlist

If you only revise ten things, revise these:

1. **DokLink race condition** - the story and the fix.
2. **Why React Native over Flutter.**
3. **ACID properties** - all four, with the DokLink tie-in.
4. **Binary search, BFS, DFS, doubly linked list** - be able to write them (part 4).
5. **Hashmaps** - what and why, $O(1)$ average lookups.
6. **Inheritance and the four OOP pillars.**
7. **JWT, OTP, HMAC-SHA256, XSS vs CSRF** - one line each.
8. **RAG and what FloatChat does** - your Joule bridge.
9. **Why STAR** and **Why SAP** - said with real enthusiasm.
10. **S-A-P**, never “sap.” And have your **questions for them** ready.